Today's Topics:

- Why GPU Computing?
- CPU vs GPU
- Introduction to CUDA
- CUDA Programming Basics
- Memory in CUDA



Computing Capability of Systems

- Evolution of Technology:
  - Vacuum Tube -> Transistor -> Integrated Circuit (IC) -> Very Large Scale Integration (VLSI)
- Performance Improvements:
  - Processor speed has been doubling approximately every 1.5 years due to advances in technology and hardware organization.
- Challenges:
  - Increasing clock rates face the issue of power dissipation.
- New Approach:
  - Adoption of multiple cores to overcome power dissipation challenges.
  - Multicore chips are capable of processing tasks in parallel, enhancing performance and efficiency.

CPUs: General-purpose processing



Why not CPU alone?

CPU is Latency Oriented Design

- High clock frequency, Large caches, Sophisticated control, Powerful ALU.
- Good for complex decisions, branching, and sequential tasks

CPU is designed for latency—fa
response for a single task

Let's assume you have to add 1 million numbers…
How will CPU do it?

CPU will process one by one
or maybe using a few cores…
but still limited parallelism

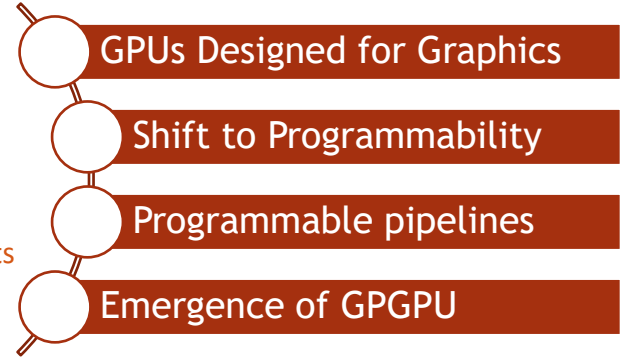
"Which task is better for CPU?"

Solving one complex problem



Rise of GPU Computing [1]

- GPUs Designed for Graphics
  - GPUs were developed solely for accelerating graphics rendering in applications
  - Image rendering, texture mapping, and 3D object transformations.
  - Early GPUs were highly specialized for graphics.
  - Couldn't be used for general-purpose computing outside of graphical tasks
- Shift to Programmability :
  - OpenGL and DirectX used to interact with a GPU and Performing arbitrary computations on a GPU would still be subject to the constraints of programming within a graphics API.
- Programmable pipelines
  - Designed to produce a color for every pixel on the screen using programmable arithmetic units (Pixel shaders).
  - Arithmetic being performed on the input colors .
  - Custom code for vertex and pixel processing.



[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

Offload Computing [1]

- Emergence of Data-Intensive Applications:
  - The growing volume and complexity of data require enhanced computing capabilities.
  - **Accelerators (**specialized hardware) are used to increase computing performance beyond what multicore technology can achieve.
  - **Accelerators Computing :** specialized hardware to perform certain types of computation more efficiently than is possible with general-purpose CPUs.
    - GPU -parallel processing\
- We don’t replace CPU. Instead, we **offload work** to other specialized hardware



Heterogeneous Parallel Computing

- CPU and GPU work together.
- GPUs: Throughput Oriented Design
  - Small caches, To boost memory throughput, Simple control (not Branch control)
  - Energy efficient ALU, Require massive number of threads to tolerate latencies
- Memory bandwidth (GPU operating at approximately six times the memory bandwidth)



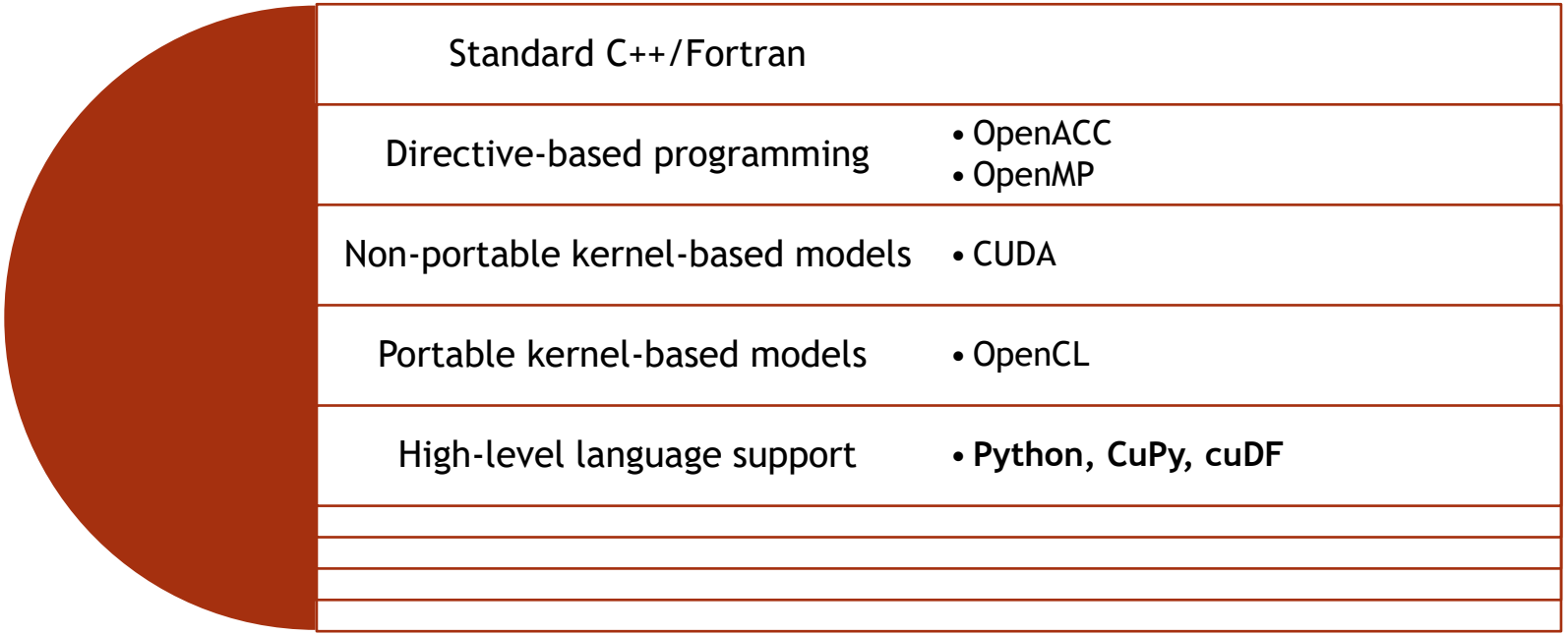
FIGURE 1.1
CPUs and GPUs have fundamentally different design philosophies.

1Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu



Programming models for GPUs

There are different ways to use GPUs for computations



CUDA is a programming model developed by NVIDIA that allows us to write programs for GPU."

GPU Architecture

- GPUs consist of Streaming Multiprocessors (SMs)
- SMs contain Streaming Processors (SPs) or Processing Elements (PEs)
  - Each SM in has a number of streaming processors (SPs) that share control logic and instruction cache.
  - Each core contains one or more ALUs and FPUs
- Two SMs form a building block;
- Architecture features
  - Number of **registers and cores**, cache and memory size, supported arithmetic instructions
  - Programmer should be aware of the differences among different versions of hardware

[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

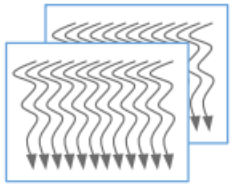


Threads



Kernel is executed by threads processed by CUDA Core

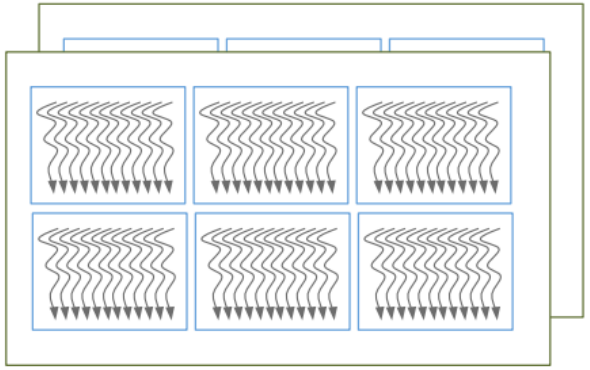
Blocks



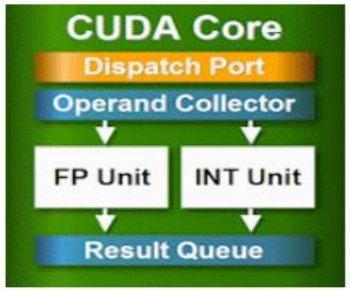
512-1024 threads / block

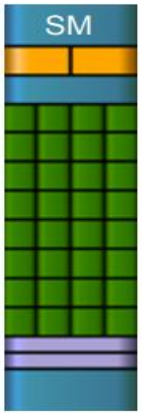
Maximum 8 blocks per SM
32 parallel threads are executed at the same time in a *WARP*

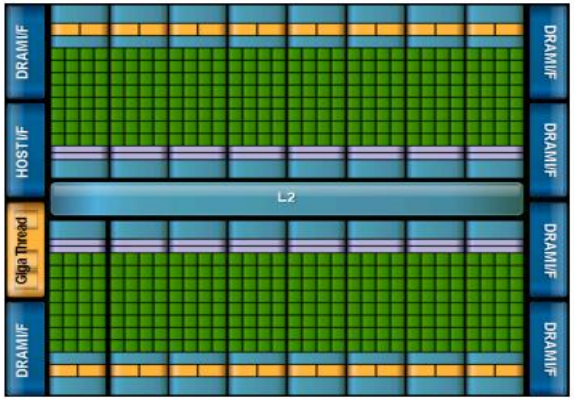
Grids



One grid per kernel with multiple concurrent kernels







PUZZLE/QA

You have 1000 data elements.
Each thread processes 1 element.
how many threads do you need?”

1000 threads

If each block has 100 threads, how many blocks

10 blocks

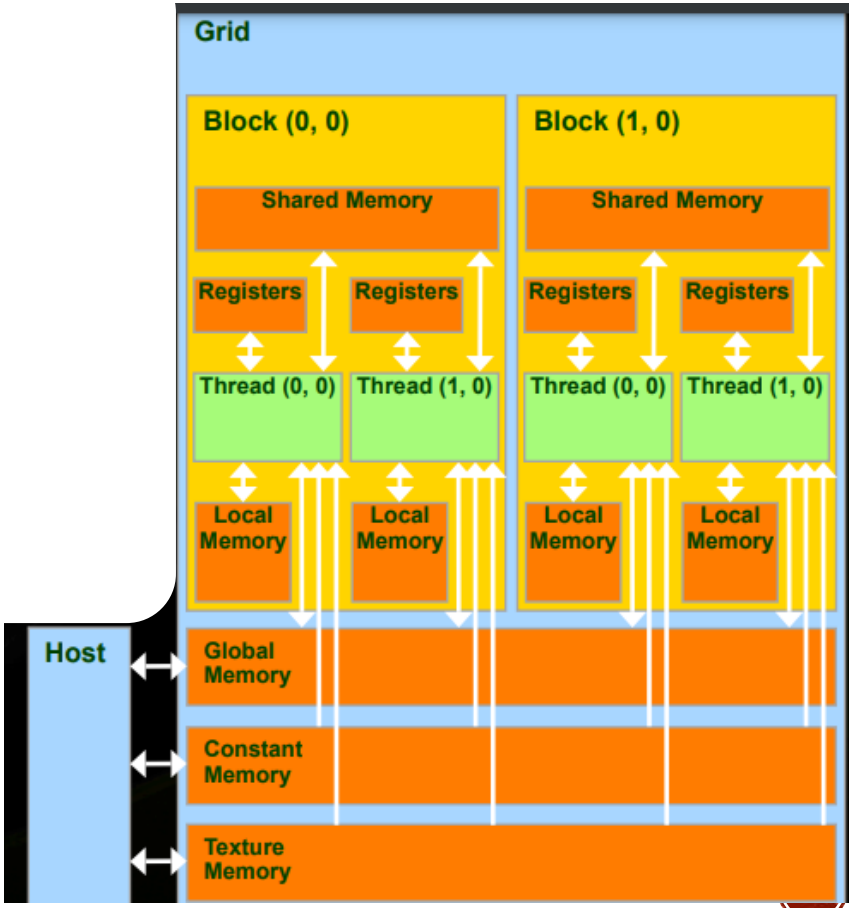
Block 0: T0 T1 T2 T3
Block 1: T0 T1 T2 T3
Block 2: T0 T1 T2 T3

- “How many blocks?”
- “How many threads per block?”
- “Total threads?”



Memory Spaces

- Registers are private to each thread [Private]
- Shared memory is shared between threads in the same block [block-level]
- Global memory is large and shared by all threads, but it is slow [All the threads]



Learned So Far

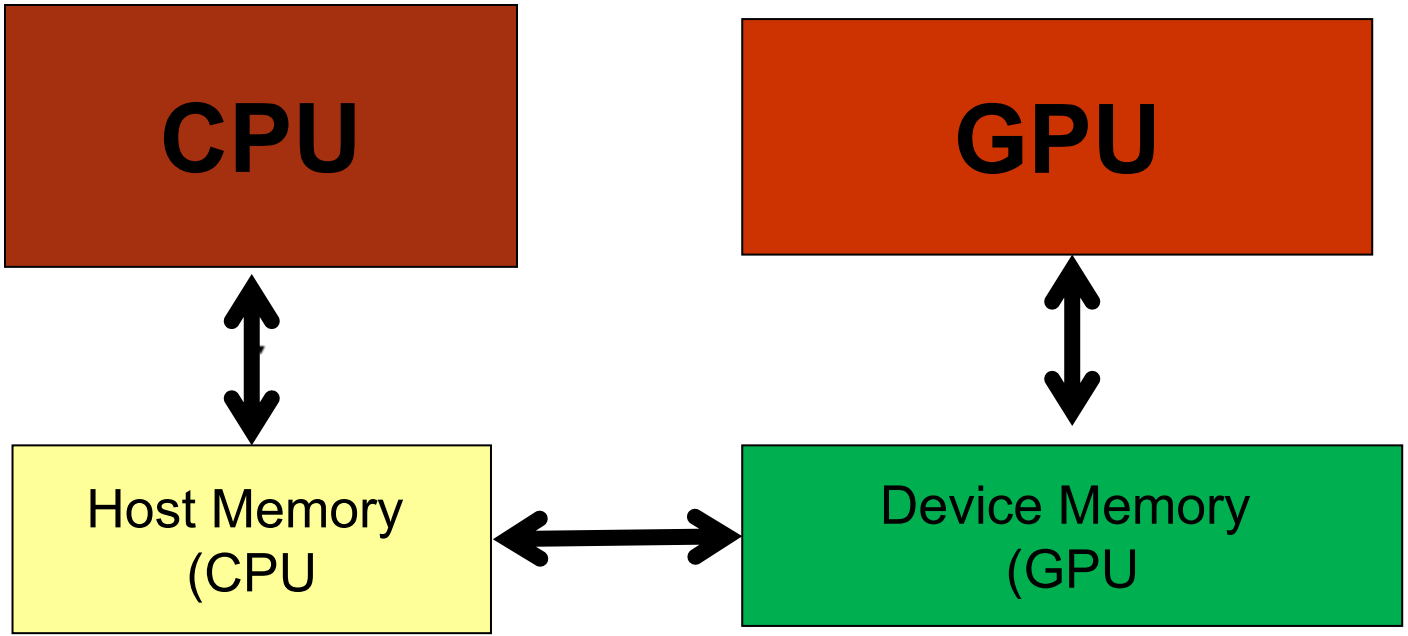
- CPU vs GPU
- Why GPU?
- GPU Architecture and Organization:
    - Threads → basic unit of work
    - Blocks → group of threads
    - Grid → group of blocks
- Memory in CUDA

Next : Cuda Programming

[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

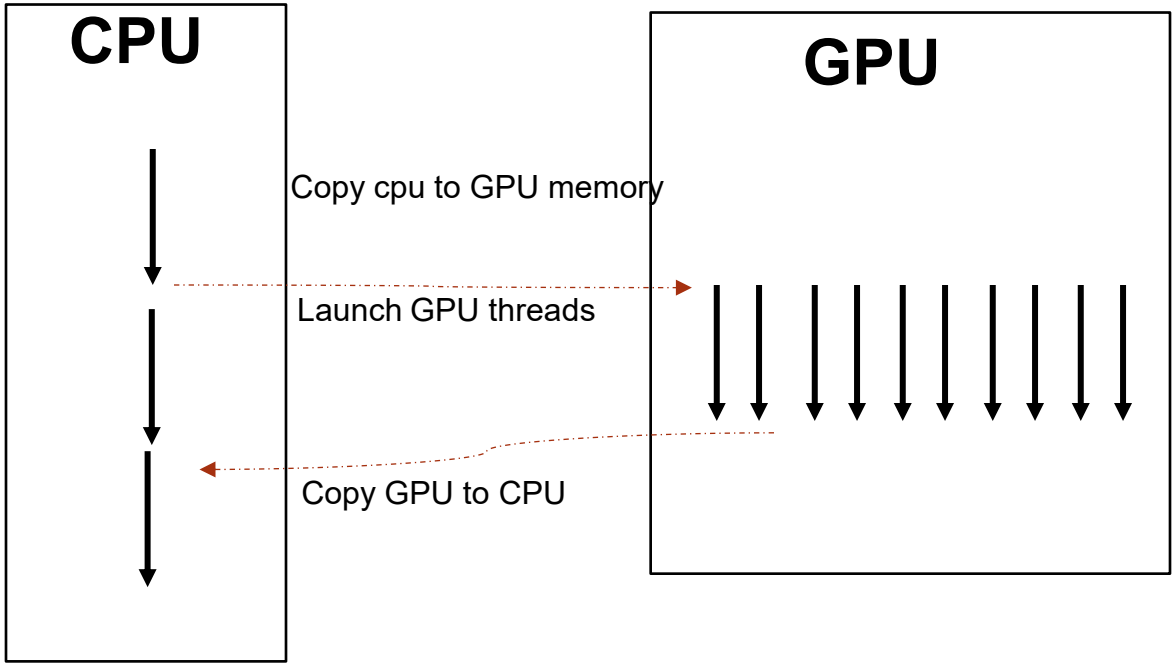


Programmer's view with GPU





Programmer's view with GPU



Not everything runs on GPU. Only parallel part goes to GPU

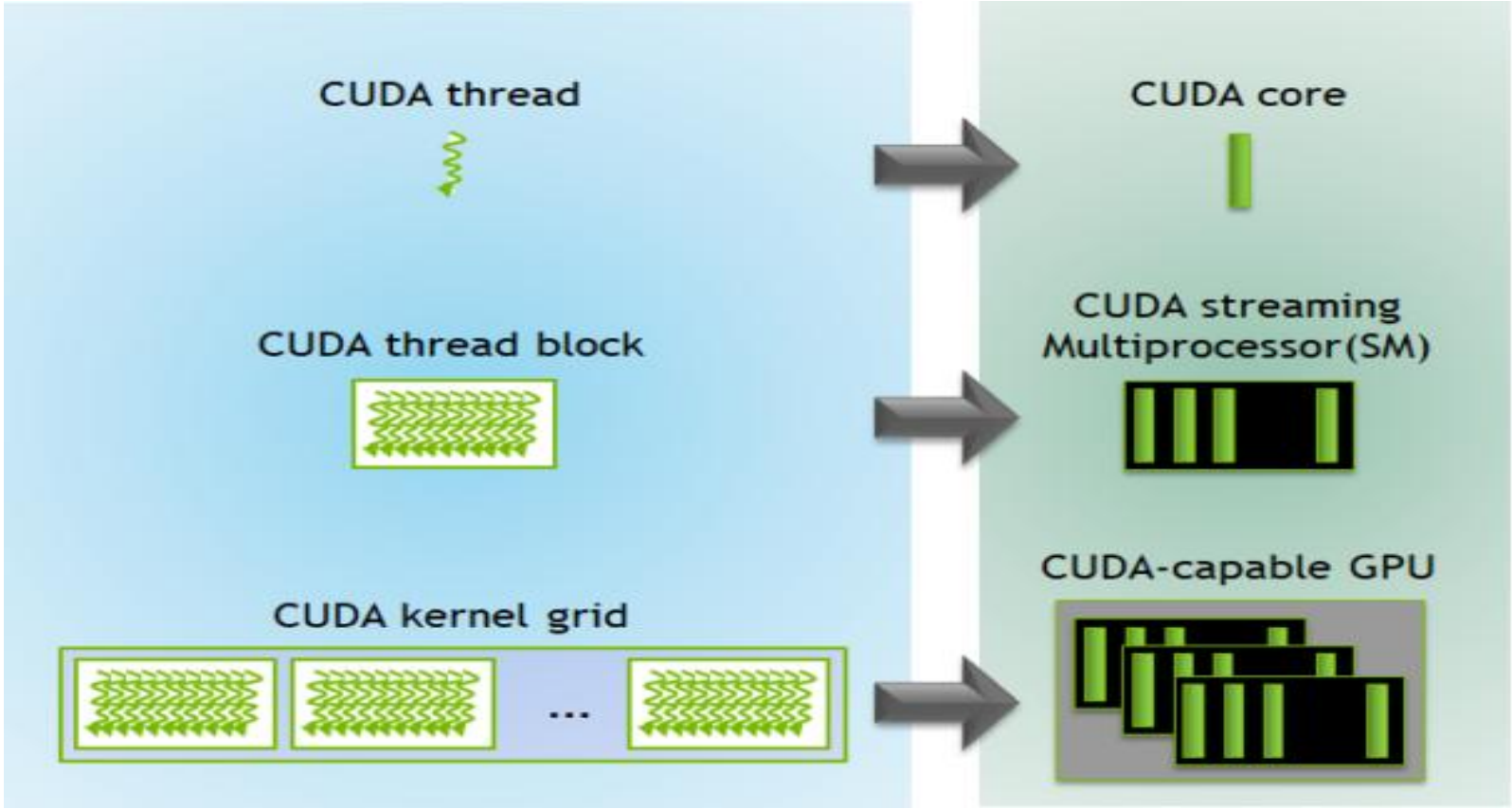


What is CUDA?

- General purpose parallel computing platform and programming model that leverages the parallel compute engine in NVIDIA GPUs
- Massively parallel, Compute Intensive.
- Key abstractions in CUDA
  - Hierarchy of thread groups
  - Shared memories
  - Barrier synchronization
- CUDA ->Single Instruction Multiple Thread
    - Computations can be divided into hundreds or thousands of independent units of work.

Compute Unified Device Architecture (CUDA)




CUDA thread
CUDA core
CUDA thread block
CUDA streaming Multiprocessor(SM)
CUDA kernel grid
...
CUDA-capable GPU

CUDA PROGRAM STRUCTURE

- A CUDA program consists of one or more phases that are executed on either the host (CPU) or a device such as a GPU.
- The phases that exhibit little or no data parallelism are implemented in host code.
- The phases that exhibit rich amount of data parallelism are implemented in the device code.
- A CUDA program is a unified source code encompassing both host and device code.



CUDA Compilation and PTX

The NVIDIA C compiler (nvcc) separates the two during the compilation process.

1. The host code is straight ANSI C code; it is further compiled with the host's standard C compilers and runs as an ordinary CPU process.
2. The device code is written using ANSI C extended with keywords for labeling data-parallel functions, called kernels
3. The device code is typically further compiled by the nvcc and executed on a GPU device

- The kernel functions (or, simply, kernels) typically generate a large number of threads to exploit data parallelism.
  - Ex: Entire matrix multiplication computation can be implemented as a kernel where each thread is used to compute one element of output matrix

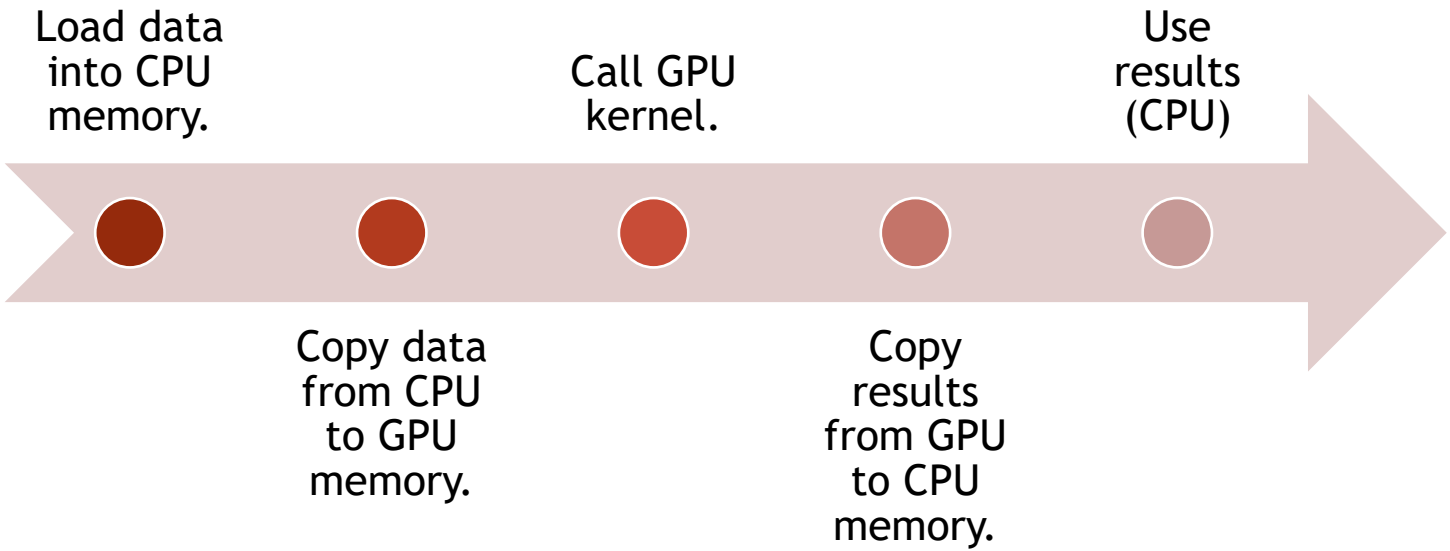


CUDA Advantages

- Random access to memory
  - Thread can access any memory location.
- Unlimited access to memory.
- Low learning curve Just a few extensions to C
- Thousands of lightweight concurrent threads (No switching overhead)



CUDA Program Flow



A First Program

```
/* Cuda Hello, World, hello.
#include <stdio.h>
__global__ void mykernel(void) {
}
int main(void){
mykernel<<<1,1>>>();
printf("Hello, World\n");
return 0;
}
```

__global__ : specifier indicates a function that runs on device (GPU)

host =>cpu

CUDA Compilation and PTX

- Compile: **nvcc** hello.cu

nvcc separates source code

- Host components
  - Device functions (e.g. mykernel())
  - NVIDIA compiler
- Device components
  - Host functions (e.g. main())
  - gcc

- Code sent from CPU to GPU is in Parallel Thread Execution **(PTX)**



CUDA Concepts and Terminology

- Kernel: function to be run on a GPU (or a device).
- << parameters >> : kernel launch with parameters

```
myKernel<<<grid, block>>>();
```

```
mykernel<<<1,1>>>();
```

- **grid** = how many blocks to launch
- **block** = how many threads per block
- athreads execute the same kernel code



RUNNING IN PARALLEL

kernel execution
configuration <<<...>>>

Informs how many threads to launch on
GPU during CUDA runtime .

<<<2,4>>> → 2 blocks × 4 threads = 8 threads

source NVIDIA



Execution Model

- **Kernels are launched in grids**
- **Blocks run in parallel**
- **Threads inside block run in parallel**



CUDA Function Declarations

- `__global__` defines a kernel function
  - Must return `void`
  - **Example:** `__global__ void KernelFunc()`
  - **Executed on the device, only callable from the host**
- `__device__` defines a function called by kernels.
  - Example: `__device__ float DeviceFunc()`
  - **Executed on the device, only callable from the device**
- `__host__` defines a function running on the host
  - **Example:** `__host__ float HostFunc()`
  - **Executed on the host, only callable from the host**



CUDA Function Declarations

- For functions executed on the device:
  - No recursion
  - No static variable declarations inside the function
  - No variable number of arguments



CUDA Threads

- A block can be split into parallel threads

Using blocks:

```
__global__ void add(int *a, int *b, int *c){
   c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

Using threads all in one block:

```
__global__ void add(int *a, int *b, int *c){
   c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}
```

- Hardwire limits the number of blocks in a single launch to 65,535.
- Hardwire also limits the number of threads per block with which we can launch a kernel. –

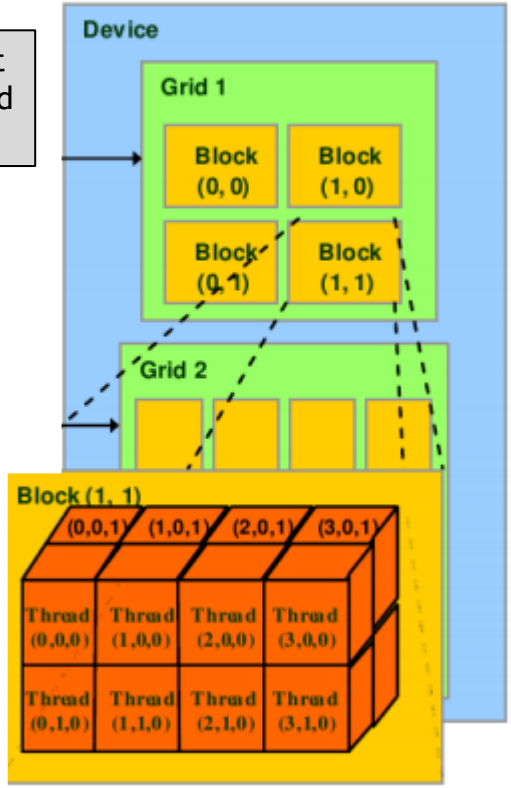
For many GPUs, maxThreadsPerBlock = 512 (or 1024, version 2.x above).

CUDA Concepts and Terminology

Threads, Blocks and Grids ??

- Logical partitioning with:
  - Threads
    - Thread IDs
  - Blocks of threads
    - Block IDs
    - Block Dimensions
    - Threads are arranged in 1D, 2D, 3D logical fashion
  - Grid of blocks
    - Grid Dimensions
    - Blocks are arranged in 1D, 2D, 3D logical fashion •
  - Each is limited with physical resources available

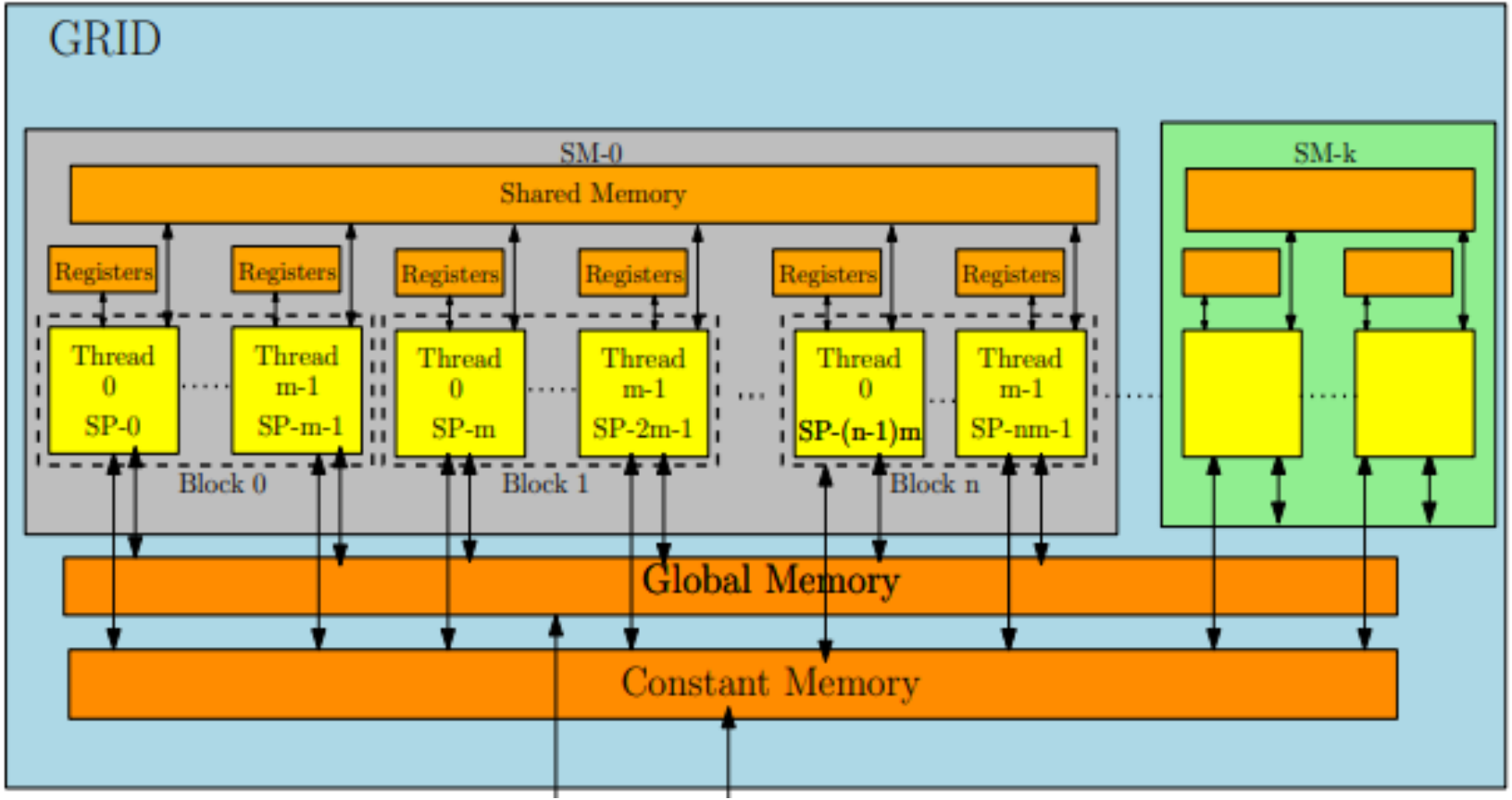
• Each SM is independent and bound by threads and blocks count



[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

source NVIDIA

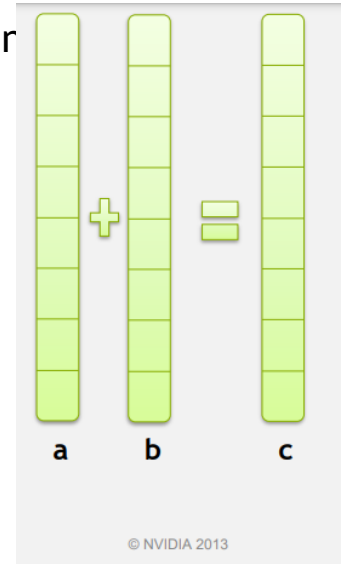



GRID
SM-0
Shared Memory
Registers
Registers
Registers
Registers
Registers
Registers
Thread 0 SP-0
Thread m-1 SP-m-1
Thread 0 SP-m
Thread m-1 SP-2m-1
Thread 0 SP-(n-1)m
Thread m-1 SP-nm-1
Block 0
Block 1
Block n
SM-k
Global Memory
Constant Memory

Built-in variable “blockIdx”

- blockIdx; this is one of the built-in variables that the CUDA.
- It contains the value of the block index for whichever block is currently runnir device code

  - Why `blockIdx.x`?:
    - CUDA C allows you to define a group of blocks in two dimensions. : .x and .y
    - matrix math or image processing
  - collection of parallel blocks a *grid*.



By using blockIdx.x to index into the array, each block handles a different index

source NVIDIA



BLOCK 1

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 0;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 2

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 1;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 3

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 2;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

BLOCK 4

```
__global__ void
add( int *a, int *b, int *c ) {
    int tid = 3;
    if (tid < N)
        c[tid] = a[tid] + b[tid];
}
```

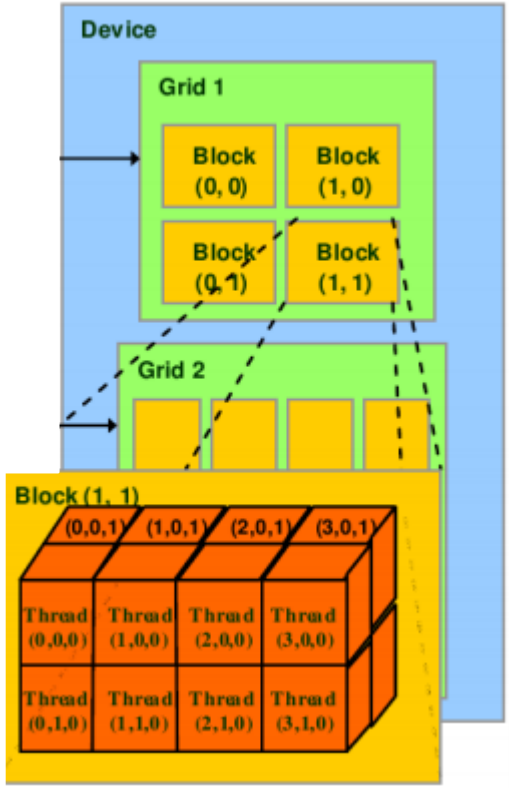
four blocks, all running through the same copy of the device code but having different values for the variable blockIdx.x.

source NVIDIA



SM, SP, Block and thread

- Thread block max size : 1024 (modern archs 2048
- SM can store max 1024 "thread contexts
- can have much less than 1024 SP
- One block in one SM
- One SM can have multiple block



If SM can store max 1024 "thread contexts", and block size is 256, we have 4 blocks per SM

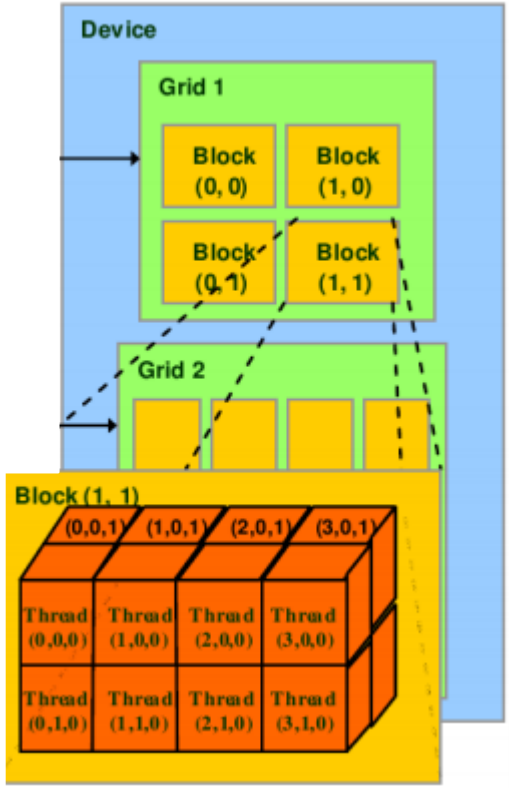
[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

source NVIDIA



SM, SP, Block and thread

- GPU HW schedule :
  - The hw scheduler decided which threads to map to a collection of SPs in SIMD fashion :: SIMT model of execution.
  - This collection is physically guaranteed to execute in parallel
- The unit of such collections is "**warp**"



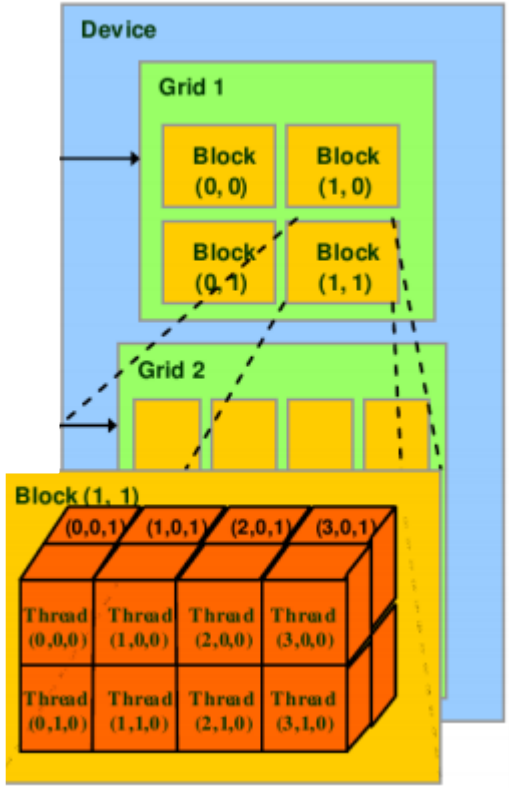
[1]Programming Massively Parallel Processors,2rd ed. By David B. Kirk Wen-mei W. Hwu

source NVIDIA



SM, SP, Block and thread

- The unit of such collections is "**warp**"
- Warp is a unit of thread Scheduling in SMs
- Warp size is implementation specific (typically 32 threads
- Warps are executed in an SIMD fashion
- the warp scheduler launches warps of threads and each w**arp typically executes one instruction across parallel thread**

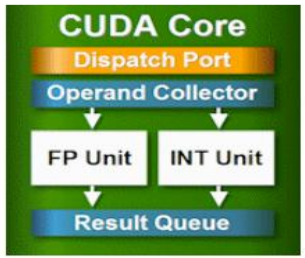


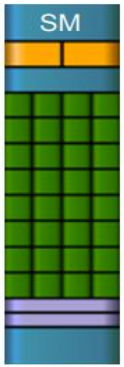
If a SM has 128 SPs, it can execute 4 Warps at a given time (one Warp has 32 Threads

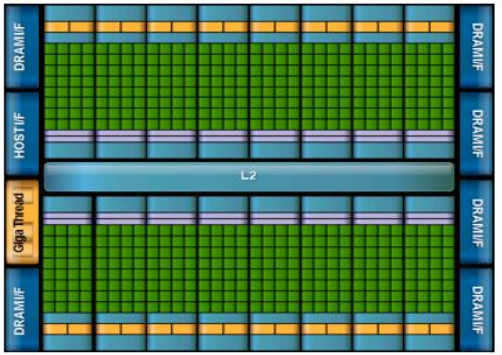
Thread-block Scheduling in SM

Warp Scheduling in SM

- Once a thread block is launched on a SM, all of its warps are resident until their execution finished.
- If one or both of its operands are not ready (e.g. have not yet been fetched from global memory), **a process called 'context switching' takes place** which transfers control to another warp.
- While **switching away from a particular warp,** all the data of that warp remains in the register file so that it can be quickly resume.
- If more than one warp are eligible for execution in a SM
  - ▶ Round Robin (RR) , Least Recently Fetched (LRF) etc







[1]Programming Massiv

- Using blocks:

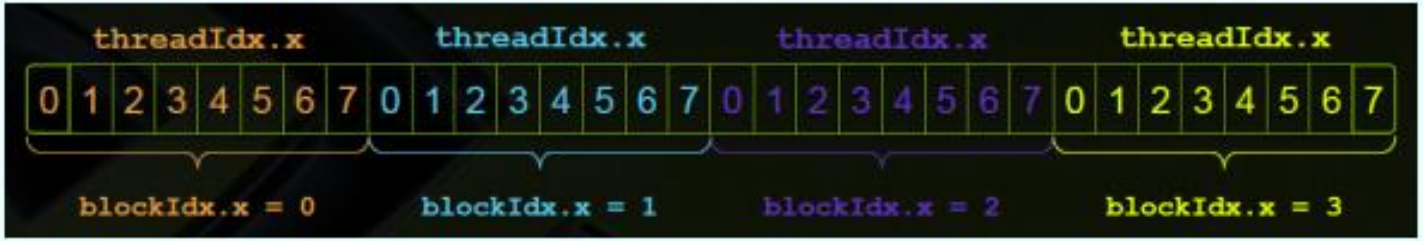
```
__global__ void add(int *a, int *b, int *c){
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

- Using threads all in one block:

```
__global__ void add(int *a, int *b, int *c){
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}
```



- **Vector addition to use both blocks and threads**
  - Consider indexing an array with one element per thread
  - We also use 8 threads per block.
  - With “M” threads/block a unique index for each thread is given by:
  - int index = threadIdx.x + blockIdx.x*M;




```
__global__ void add(int *a, int *b, int *c) {
int index = threadIdx.x + blockIdx.x * blockDim.x;
c[index] = a[index] + b[index];
}
```

Source: https://giahuy04.medium.com/using-gpu-memory-a75e0fabe3f0

Addition on the Device

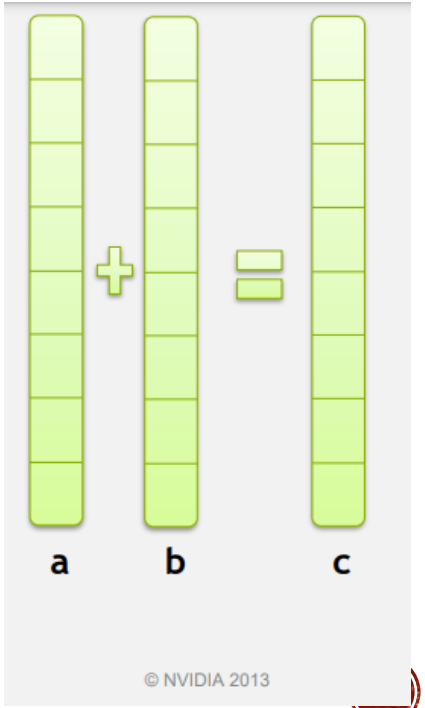
- A simple kernel to add two integers

```
__global__ void add( int a, int b, int *c ) {
    *c = a + b;
}
```

add() will execute on the device

add() will be called from the host

- **a, b and c** must point to device memory as add() runs on the device
- **How do we allocate device memory ?**
- Memory Management ?
  - Device pointers
  - Host pointers



source NVIDIA

Memory Management

- cudaMalloc(),
- cudaFree(),
- cudaMemcpy()

- Device pointers point to GPU memory
  May be passed to/from host code
  May not be dereferenced in host code

- Host pointers point to CPU memory
  May be passed to/from device code
  May not be dereferenced in device code

Restrictions on the usage of device pointer as follows:

- You *can* pass pointers allocated with `cudaMalloc()` to functions that execute on the device.
- You *can* use pointers allocated with `cudaMalloc()` to read or write memory from code that executes on the device.
- You *can* pass pointers allocated with `cudaMalloc()` to functions that execute on the host.
- You *cannot* use pointers allocated with `cudaMalloc()` to read or write memory from code that executes on the host.



cudaMemcpy()

- Host pointers can access memory from host code,
- Device pointers can access memory from device code.
- How to access the memory on device from host code?

    - Using cudamemcpy()
    - An additional parameter to specify which of the source and destination pointers point to device memory.
        - cudaMemcpyDeviceToHost: Source pointer is a device pointer and the destination pointer is a host pointer.
        - cudaMemcpyHostToDevice: source data is on the host and the destination is an address on the device.

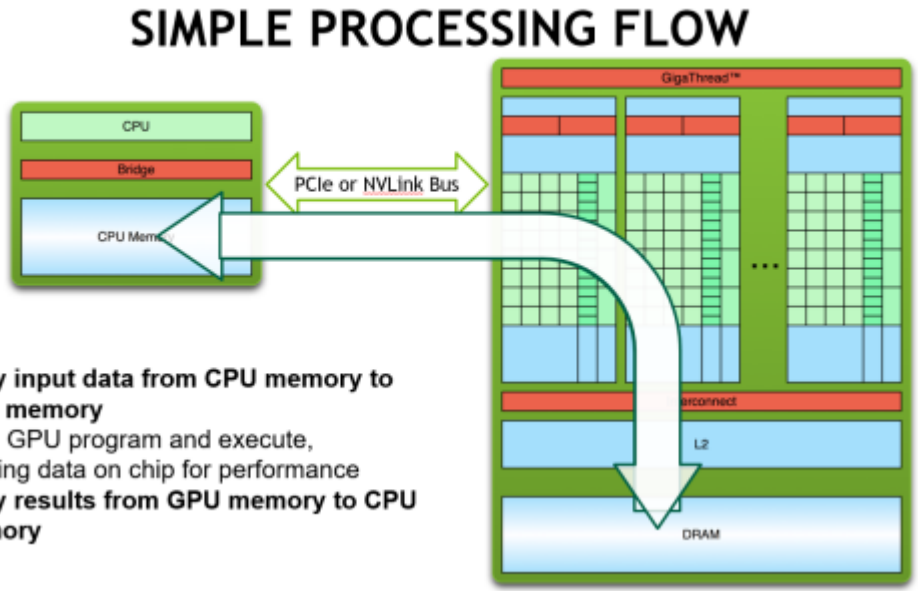


- Host pointers can access memory from host code,
- Device pointers can access memory from device code.
- How to access the memory on device from host code?

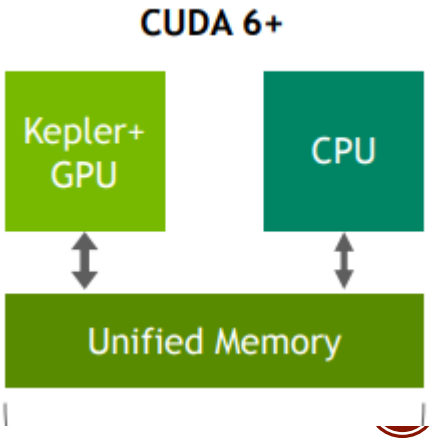
    - Using cudamemcpy()
    - An additional parameter to specify which of the source and destination pointers point to device memory.
        - cudaMemcpyDeviceToHost: Source pointer is a device pointer and the destination pointer is a host pointer.
        - cudaMemcpyHostToDevice: source data is on the host and the destination is an address on the device.



CUDA 8+: UNIFIED MEMORY

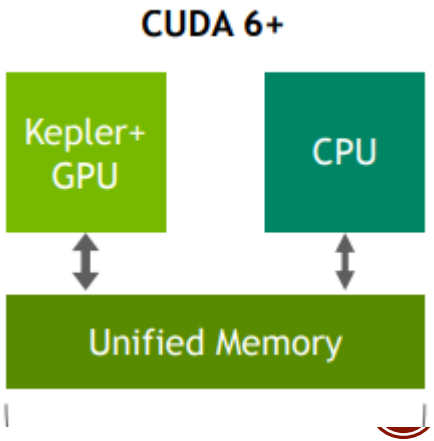


1. **Copy input data from CPU memory to GPU memory**
2. Load GPU program and execute, caching data on chip for performance
3. **Copy results from GPU memory to CPU memory**



CUDA 8+: UNIFIED MEMORY

- Single allocation, single pointer, accessible anywhere
- Eliminate need for explicit copy Simplifies code porting single allocation,
- single pointer, accessible anywhere Eliminate need for explicit copy Simplifies code porting



CUDA 8+: UNIFIED MEMORY

Ordinary CUDA Code

```
void sortfile(FILE *fp, int N) {
  char *data, *d_data;
  data = (char *)malloc(N);
  cudaMalloc(&d_data, N);
  fread(data, 1, N, fp);
  cudaMemcpy(d_data, data, N, …); // 1
  qsort<<<...>>>(data,N,1,compare); // 2
  cudaMemcpy(data, d_data, N, …); // 3

  use_data(data);
  cudaFree(d_data);
  free(data);
}
```

CUDA Code with Unified Memory

```
void sortfile(FILE *fp, int N) {
  char *data;
  cudaMallocManaged(&data, N);

  fread(data, 1, N, fp);

  qsort<<<...>>>(data,N,1,compare);
  cudaDeviceSynchronize();

  use_data(data);

  cudaFree(data);
}
```



CUDA 8+: UNIFIED MEMORY

Ordinary CUDA Code

```
void sortfile(FILE *fp, int N) {
  char *data, *d_data;
  data = (char *)malloc(N);
  cudaMalloc(&d_data, N);
  fread(data, 1, N, fp);
  cudaMemcpy(d_data, data, N, …); // 1
  qsort<<<...>>>(data,N,1,compare); // 2
  cudaMemcpy(data, d_data, N, …); // 3

  use_data(data);
  cudaFree(d_data);
  free(data);
}
```

CUDA Code with Unified Memory

```
void sortfile(FILE *fp, int N) {
  char *data;
  cudaMallocManaged(&data, N);

  fread(data, 1, N, fp);

  qsort<<<...>>>(data,N,1,compare);
  cudaDeviceSynchronize();

  use_data(data);

  cudaFree(data);
}
```



Addition on the Device: main()

```
#include <iostream>
#include "book.h"

__global__ void add( int a, int b, int *c ) {
    *c = a + b;
}

int main( void ) {
    int c;
    int *dev_c;
    HANDLE_ERROR( cudaMalloc( (void**)&dev_c, sizeof(int) ) );

    add<<<1,1>>>( 2, 7, dev_c );

    HANDLE_ERROR( cudaMemcpy( &c,
                              dev_c,
                              sizeof(int),
                              cudaMemcpyDeviceToHost ) );
    printf( "2 + 7 = %d\n", c );
    cudaFree( dev_c );

    return 0;
}
```

memory on the device

Allocate space for device copies

Copy result back to host

source NVIDIA

source NVIDIA



```
#include <stdio.h>
#include <cuda.h>
const char *msg = "Hello World.\n";
__global__ void dkernel() {
    printf(msg);
}
int main() {
    dkernel<<<1, 32>>>();
```

- A variable in CPU memory cannot be accessed directly in a GPU kernel
- A programmer needs to maintain copies of variables.
- It is programmer's responsibility to maintain sync.

error: identifier "msg" is undefined in device code



References

- NVIDIA Practices_Guide 2023
- NVIDIA Programming_Guide 2023
- https://www.int.washington.edu/PROGRAMS/12-2c/week3/clark_01.pdf
- https://cuda-utorial.readthedocs.io/en/latest/tutorials/tutorial01/
- https://developer.nvidia.com/blog/even-easier-introduction-cuda/

